Transformari Geometrice 2D

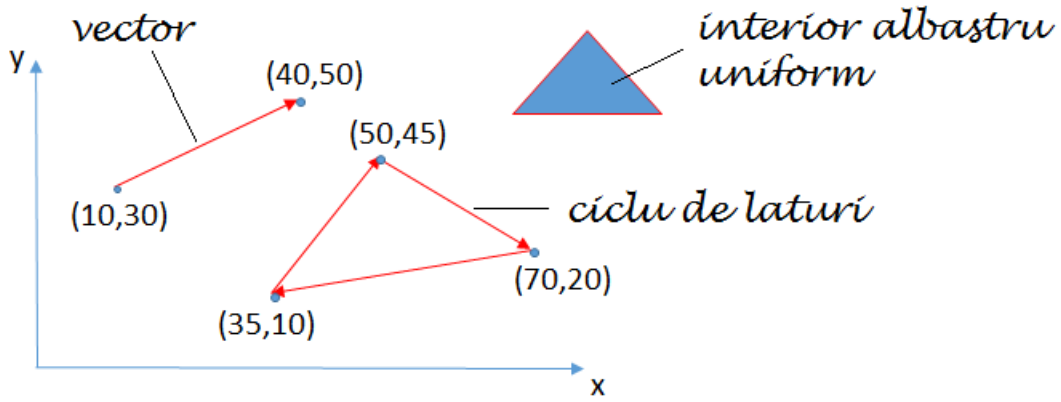
Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Transformari geometrice(1)

Obiectele 2D/3D sunt reprezentate prin:

- Date geometrice: coordonatele varfurilor, raportate la un sistem de coordonate carteziene 2D sau 3D;
- Atribute topologice, care definesc modul in care sunt conectate varfurile (laturi, ciclul de laturi al unei feţe, ş.a.);
- Atribute de aspect: culoarea, tipul de interior pentru suprafete 2D, atribute de material pentru suprafete 3D (ex: reflexia/refractia luminii de catre suprafata), s.a.



Transformarile geometrice se aplica (coordonatelor) varfurilor obiectului si nu afecteaza atributele sale!

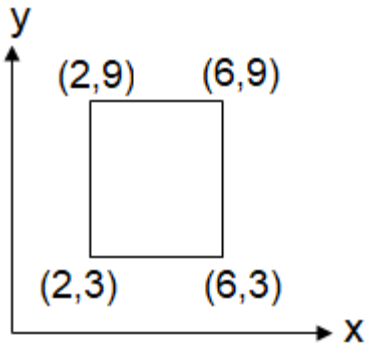
Transformari geometrice (2)

- Sunt operatii fundamentale in sinteza imaginilor
- Folosite pentru:
  - Redarea desenelor la diferite marimi
  - Compunerea desenelor sau a scenelor 3D
  - Realizarea animatiei
  - Transformarea obiectelor dintr-un spatiu de coordinate logic, in care sunt definite, in spatiul fizic de afisare
  - Etc.

Transformari geometrice 2D (1)

Sunt transformari ale obiectelor definite in sistemul de coordonate carteziene XOY, numite şi obiecte 2D. Fiecare vârf al unui obiect 2D este definit printr-o pereche de coordonate (x,y).

Transformari geometrice elementare



- Translaţia
- Scalarea faţă de origine
- Rotaţia faţă de origine
- Forfecarea faţă de origine
- Oglindirile faţă de axele principale şi faţă de origine

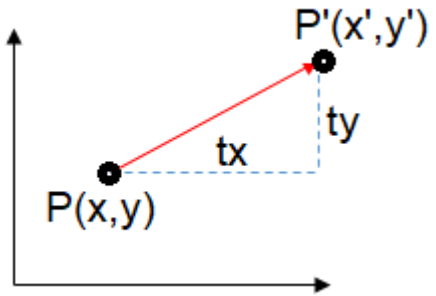
Orice alta transformare se obtine prin compunrea a 2 sau mai multe transformari elementare

- Scalarea /rotaţia fata de un punct oarecare din plan
- Scalare compusa cu translatie, etc

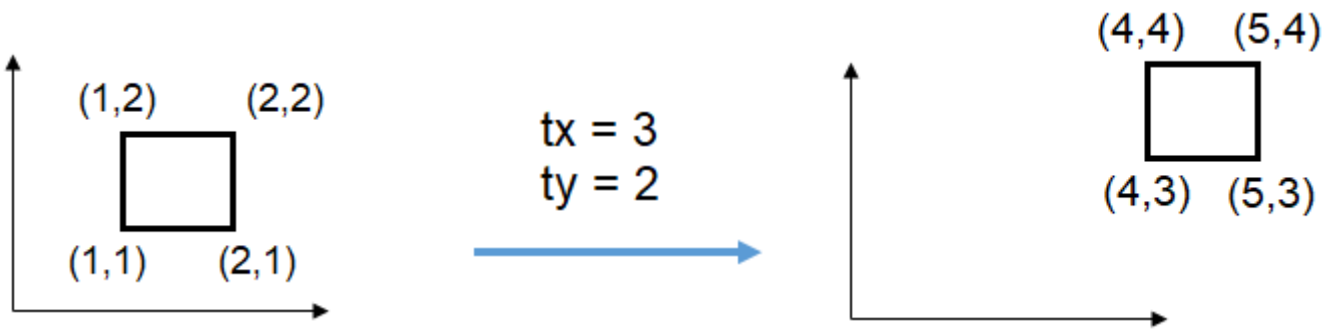
Transformari geometrice 2D elementare (1)

- Translatia (1)

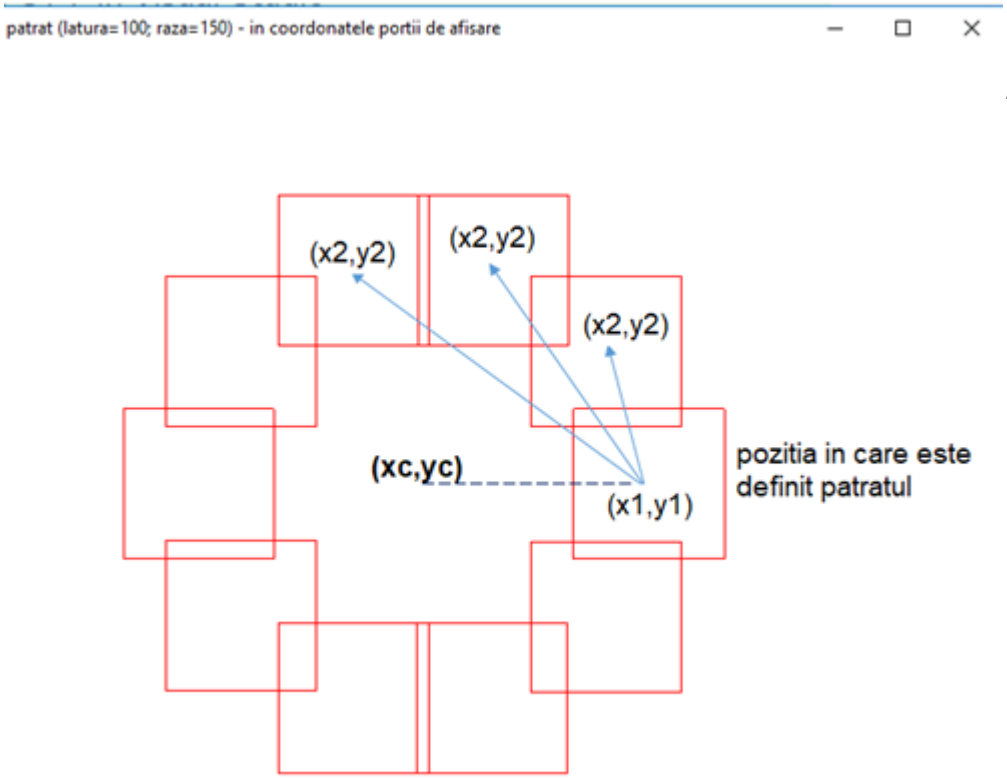
  Este definita printr-un vector, T[tx,ty].

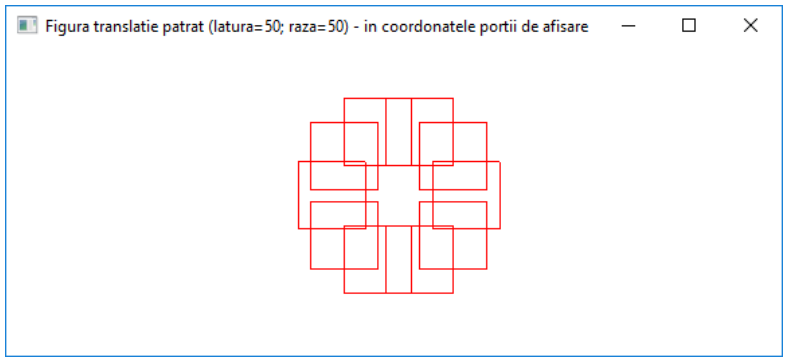


$$x' = x + tx$$
$$y' = y + ty$$



Translatia unui patrat pe circumferinta unui cerc (1)






```
//Campuri ale clasei DesenTranslatiePatrat
// initializate in functia Init
 int raza, xc, yc, latura; Mesh *square;

void DesenTranslatiePatrat::Update(float  Seconds)
{ //functia deseneaza cadrul imagine curent
   glm::ivec2 resolution = window->GetResolution();
   xc = resolution.x / 2;
   yc = resolution.y / 2;

   float x1 = xc + raza, y1 = yc, x2, y2;
   for (int i = 0; i < 10; i++)
   { // distanta unghiulara intre puncte: 6.28/10
        x2 = xc + raza * cos(0.628f*i);
        y2 = yc + raza * sin(0.628f *i);

     // seteaza matricea de transformare a varfurilor
     modelMatrix =
                Transform2D::Translate(x2 - x1, y2 - y1);

     // se cere desenarea patratului
      RenderMesh2D(square,
                shaders["VertexColor"], modelMatrix);
   }
 }
```

Translatia unui patrat pe circumferinta unui cerc (2)

```
void DesenTranslatiePatrat::Init()
{
glm::ivec2 resolution = window->GetResolution();

// initializare camera
……

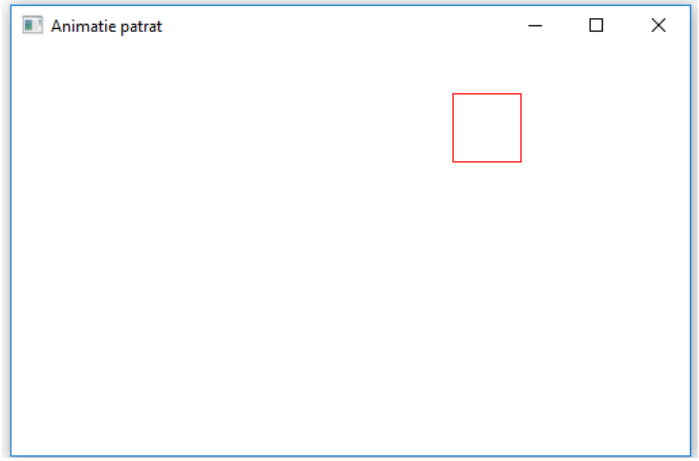
latura = 100;
raza = 150;
xc = resolution.x / 2; yc = resolution.y / 2;

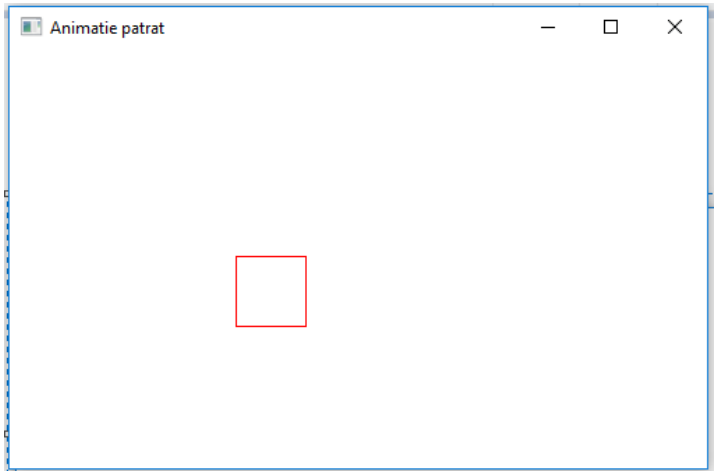
//pozitia coltului stanga-jos al patratului
glm::vec3 corner = glm::vec3(xc + raza - latura / 2, yc - latura / 2,0);

square = Object2D::CreateSquare("square", corner, latura, glm::vec3(1, 0, 0));
AddMeshToList(square);

}
```

Animatia translatiei patratului






```
//campuri ale clasei AnimatieTranslatiePatrat initializate in
//functia Init la fel ca in clasa DesenTranslatiePatrat
  int raza, xc, yc,latura, angle; Mesh * square;

void AnimatieTranslatiePatrat::Update(float deltaSeconds)
{ //functia deseneaza cadrul imagine curent

   glm::ivec2 resolution = window->GetResolution();
   xc = resolution.x / 2;
   yc = resolution.y / 2;
   float x1 = xc + raza, y1 = yc, x2, y2;

   angle += 0.5 * deltaSeconds;

   x2 = xc + raza * cos(angle); y2 = yc + raza * sin(angle);

    // seteaza matricea de transformare a varfurilor

    modelMatrix = Transform2D::Translate(x2 - x1, y2 - y1);

   //  se cere desenarea patratului translatat cu unghiul angle

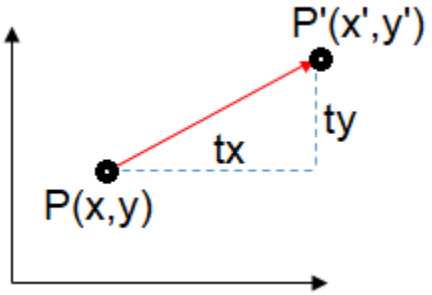
  // pe circumferinta cercului

   RenderMesh2D(square,

                    shaders["VertexColor"], modelMatrix);
}
```

Transformari geometrice 2D elementare (2)

- Translatia (2)



$$x' = x + tx$$
$$y' = y + ty$$

Se doreste o reprezentare matriciala a transformarii, pentru compunerea sa cu alte transformari: compunerea transformarilor se realizeaza prin inmultirea matricilor.

Un punct din plan, P(x,y), se reprezintă în coordonate carteziene printr-un vector [x, y] sau $\begin{bmatrix} x \\ y \end{bmatrix}$

→ Nu exista o matrice M, de 2x2, astfel incat sa exprimam translatia prin:

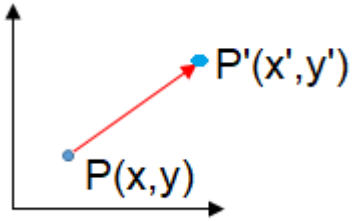
[x', y'] = [x, y] * M sau $\begin{bmatrix} x' \\ y' \end{bmatrix} = M * \begin{bmatrix} x \\ y \end{bmatrix}$

Transformari geometrice 2D elementare (3)

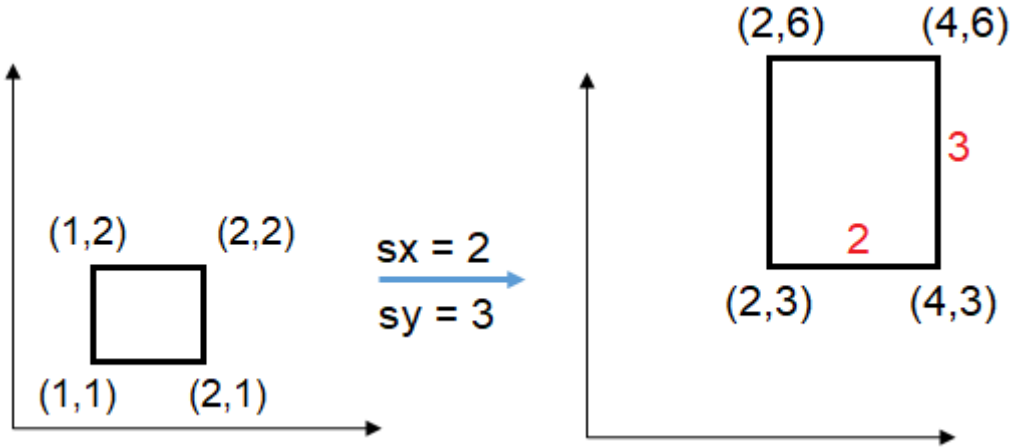
Scalarea fata de origine

Este definita prin 2 numere reale, de regula pozitive:

- sx - scalarea pe axa OX
- sy - scalarea pe axa OY



x' = x * sx
y' = y * sy



Efecte:

- Daca sx sau sy sau ambii sunt >1:.....
- Daca sx sau sy sau ambii sunt <1:.....
- ➢ sx = sy , scalare uniforma

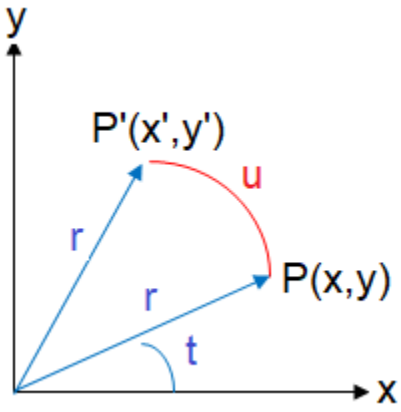
Reprezentarea matriciala

$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformari geometrice 2D elementare (3)

Rotaţia fata de origine



x = r*cos(t)
y = r*sin(t)

Relatia dintre coordonatele carteziene si coordonatele polare ale unui punct P(x,y)

x' = r*cos(t+u) = r*(cos(t)*cos(u) – sin(t)*sin(u)) = x*cos(u) – y*sin(u)
y' = r*sin(t+u) = r*(cos(t)*sin(u) + sin(t)*cos(u)) = x*sin(u) – y*cos(u)

Rotatia fata de origine

x' = x*cos(u) - y*sin(u)
y' = x*sin(u) + y*cos(u)

$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) \\ \sin(u) & \cos(u) \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformari geometrice 2D fata de un punct oarecare din plan(2)

Scalarea fata de un punct oarecare din plan

- Punctul fix al transformării este un punct oarecare F(xf,yf) – coordonatele sale nu se modifica prin aplicarea transformării.
- P(x,y) este punctul transformat
- Scalarea se aplica vectorului FP [x-xf, y-yf]:

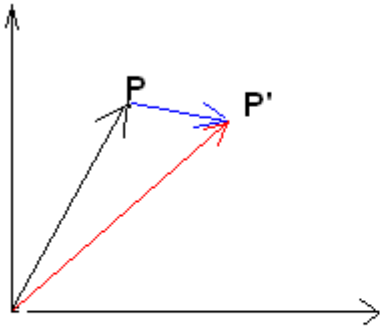
x’ – xf = sx*(x-xf)

y’ – yf = sy*(y – yf)

Rezultă:

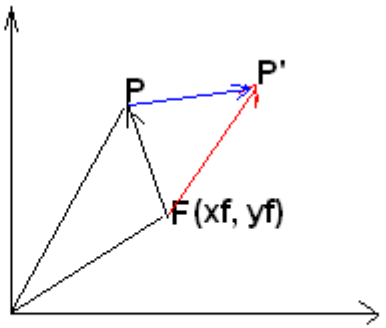
x’ = x*sx + xf –xf*sx

y’ = y*sy + yf –yf*sy



Scalarea fata de origine

x' - 0 = sx (x - 0)
y' - 0 = sy (y - 0)



Scalarea fata de un punct F din plan

x' - xf = sx (x - xf)
y' - yf = sy (y - yf)

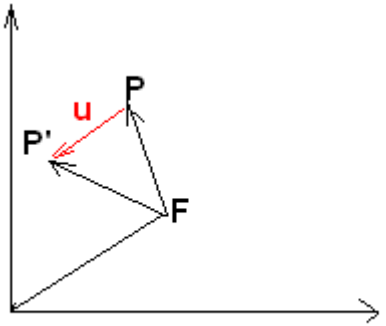
Nu poate fi exprimata în coordonate carteziene printr-o matrice de 2x2!

Transformari geometrice 2D fata de un punct oarecare din plan(2)

Rotatia fata de un punct oarecare din plan

- Punctul fix al transformarii este F(xf,yf)
- P(x,y) este punctul transformat
- Rotatia se aplica vectorului FP [x-xf, y-yf], in jurul lui F:

$$x' - xf = (x-xf)* \cos(u) - (y - yf) * \sin(u)$$
$$y' - yf = (x-xf)* \sin(u) + (y - yf) * \cos(u)$$

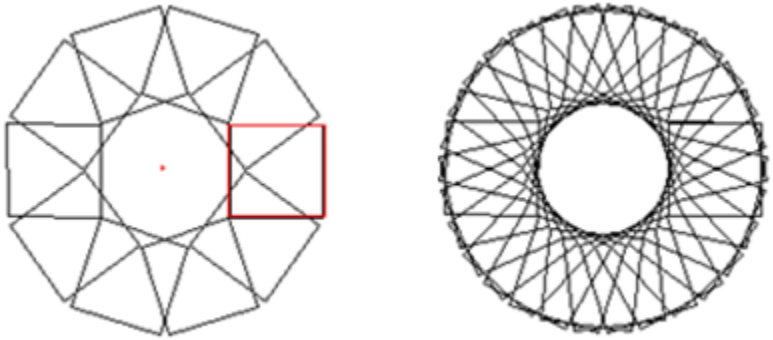


Rotatia fata de un punct F din plan

Rezultă:

$$x' = x*\cos(u) - y*\sin(u) + xf - xf*\cos(u) + yf*\sin(u)$$
$$y' = x*\sin(u) + y*\cos(u) + yf - xf*\sin(u) - yf*\cos(u)$$

Nu poate fi exprimata in coordonate carteziene printr-o matrice de 2x2!


Rotatia unui patrat in jurul unui punct din plan

Compunerea transformarilor geometrice 2D

De ce este necesară?

- Pentru a putea aplica o singura transformare care înglobeaza o secvenţă de transformari elementare, în locul aplicării unei succesiuni de transformări elementare; de ex., se aplică tuturor vârfurilor o transformare care înglobează scalare (S), rotatie (R) si translatie (T) în loc să se aplice fiecarui varf secventa de transformari elementare

  Pt = M * P în loc de P' = S * P; P" = R * P'; Pt = T * P" unde M = T * R * S

- Matricea unei transformari compuse se obtine prin înmultirea matricilor transformarilor elementare. Exemplu:

RS # SR

SR = S*R =

$$\begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

RS = S*R =

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

- **Translatia nu poate fi reprezentata în coordonate carteziene printr-o matrice!**
- **Aceasta impune reprezentarea transformarilor în coordonate omogene.**

Reprezentarea transformarilor geometrice 2D în coordonate omogene(1)

Coordonate omogene:

Un punct din plan, P(x,y), se reprezinta in coordonate omogene printr-un vector

[xw, yw, w] sau $\begin{bmatrix} xw \\ yw \\ w \end{bmatrix}$ xw = x * w; yw = y * w; w – orice numar real

Exemplu: P(2, 0.5) → [2, 0.5, 1], [4, 1, 2], [20, 5, 10]

Transformarea din coordonate omogene in coordonate carteziene:

[xw yw w] → P(x, y), unde:

- pentru w #0, x = xw/w, y = yw/w
- pentru w =0, P este un punct la infinit

Reprezentarea transformarilor elementare în coordonate omogene

Translația

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

In OpenGL se folosesc vectori coloană pentru a reprezenta puncte din plan sau din spaţiu.

Scalarea față de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotația fața de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} \cos(u) & \sin(u) & 0 \\ -\sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) & 0 \\ \sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformarile inverse ale transformarilor elementare

T(tx, ty): matricea translatiei, in coordonate omogene

S(0, 0, sx, sy): matricea scalarii fata de origine, in coordonate omogene

R(0,0,u): matricea rotatiei fata de origine, in coordonate omogene

Transformarile inverse sunt:

$T(tx, ty)^{-1} = T(-tx, -ty)$

$S(0, 0, sx, sy)^{-1} = S(0, 0, 1/sx, 1/sy)$

$R(0,0, u)^{-1} = R(0, 0, -u)$

Transformari geometrice 2D compuse(1)

Exemple de transformari compuse: scalarea/rotatia fata de un punct oarecare din plan.

- Expresiile matematice ale scalării şi rotaţiei faţă de un punct oarecare din plan se pot obţine prin compunerea următoarelor transformări:
  - Translaţia prin care punctul fix al transformării ajunge în origine: T(-xf, -yf);
  - Scalarea / rotaţia faţă de origine: S(0,0,sx,sy)/R(0,0,u);
  - Translaţia inversă celei de la punctul 1: T(xf, yf).

Transformari geometrice 2D compuse(2)

Scalarea faţă de punctul F(xf,yf)

$$[x'\ y'\ 1] = [x\ y\ 1] \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_f & -y_f & 1 \end{bmatrix}}_{T(-xf,\ -yf)} \underbrace{\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{S(0,\ 0,\ sx,\ sy)} \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_f & y_f & 1 \end{bmatrix}}_{T(xf,\ yf)}$$

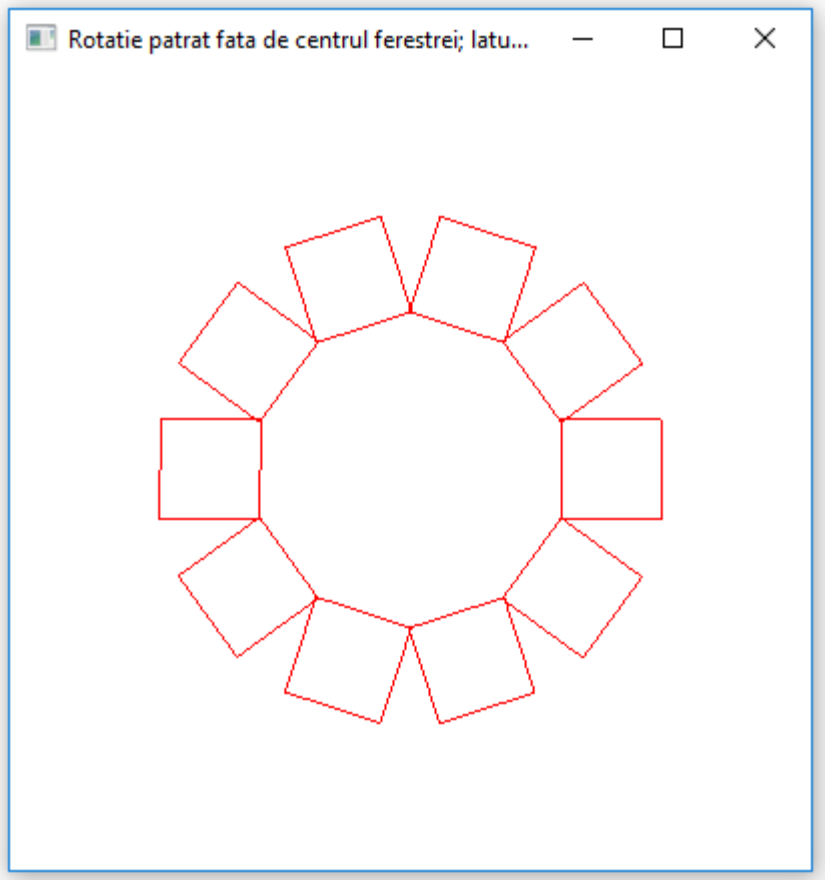
Matricea transformarii compuse:

M = T(-xf, -yf) * S(0, 0, sx, sy) * T(xf, yf)

Aplicarea transformarii compuse:

[x' y' 1] = [x y 1] * M

Rotatia unui patrat in jurul centrului ecranului




```
//campuri ale clasei DesenRotatiePatrat initializate in
//functia Init la fel ca in clasa DesenTranslatiePatrat
int raza, xc, yc, latura; Mesh* square;

 void DesenRotatiePatrat::Update(float  Seconds)
 { //deseneaza cadrul imagine curent
   glm::ivec2 resolution = window->GetResolution();
   xc = resolution.x / 2;
   yc = resolution.y / 2;

for (int i = 0; i < 10; i++)
   {
      // seteaza matricea de transformare a varfurilor
     modelMatrix = Transform2D::Translate(xc, yc) *

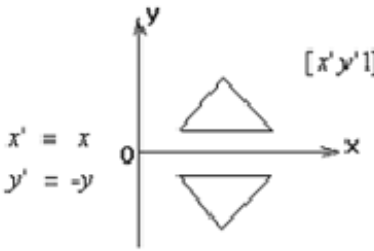
              Transform2D::Rotate(0.628f * i) *

              Transform2D::Translate(-xc, -yc);
    //  deseneaza patratul rotit fata de (xc,yc)
      RenderMesh2D(square,
                shaders["VertexColor"], modelMatrix);
   }
 }
```

Alte transformari geometrice 2D(1)

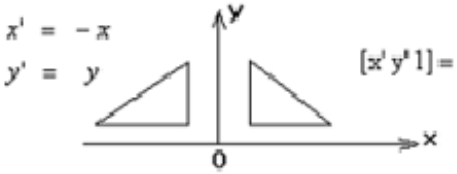
Oglindirea

Fata de axa OX



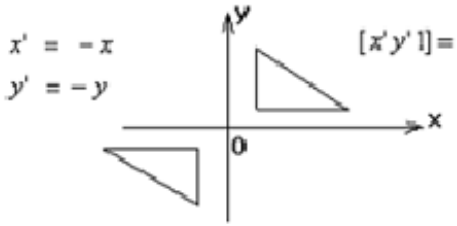
$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Fata de axa OY



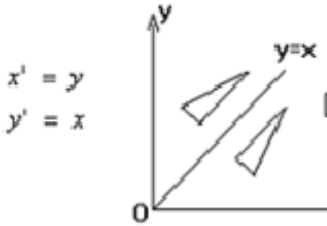
$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Fata de origine



$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Fata de dreapta x=y



$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Alte transformari geometrice 2D(2)

Oglindirea faţă de o dreaptă oarecare

Se exprimă ca transformare compusa prin înmulţirea matricilor care exprima urmatoarele transformari:

1. O translaţie, astfel încât dreapta sa treaca prin origine.

2. O rotaţie faţă de origine, a.î. dreapta să se suprapună peste una dintre axele principale.

3. Oglindirea faţă de axa principală peste care a fost suprapusă dreapta.

4. Rotaţia inversă celei de la punctul 2.

5. Translaţia inversă celei de la punctul 1.

În notaţie matricială:

M = T * R* O* R^{-1} *T^{-1} (folosind vectori linie) **sau M = T^{-1} * R^{-1} *O *R *T** (folosind vectori coloana)

Deduceti elementele matricilor T, R, O, atunci cand dreapta este data printr-un punct, (xd, yd) si o directie, D[a, b].

Alte transformari geometrice 2D(3)

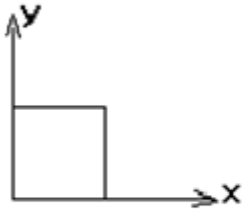
Forfecarea

Este definita prin 2 numere reale:

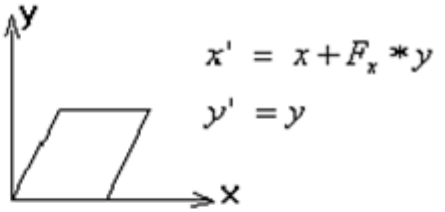
Fx: factorul de forfecare pe axa OX

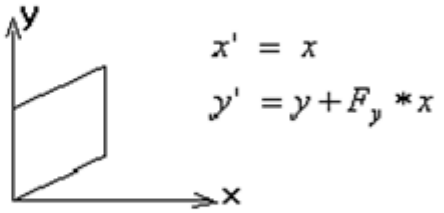
Fy: factorul de forfecare pe axa OY

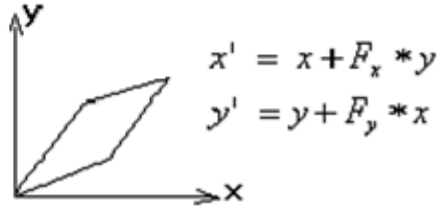
Deduceti formele matriciale ale transformarilor de forfecare



Forfecarea fata de origine







Forfecarea fata de un punct oarecare din plan, (xf,yf) , exprimata ca transformare compusa:

1. Translatie prin care punctul (xf, yf) ajunge in origine
2. Forfecarea fata de origine
3. Translatia inversa celei de la pasul 1